RISC201 Processor Project: Documentation

1. Overview

- **What it is:** a software model of the RISC201 processor (SimpleRISC Extended ISA: 32-bit instructions, 16 registers, IEEE-754 single-precision float) together with every tool needed to write, assemble, run, debug and evaluate programs for it.
- **Three processor models:** functional (one instruction at a time), **4-stage pipeline** (IF, OF, EX_MA, RW) and **6-stage pipeline** (IF, ID, OF, EX, MA, RW). The pipelines are cycle-accurate: forwarding, load-use and multi-cycle stalls, flushes and precise exceptions.
- **Microprogrammed control:** every control signal is read from a control memory, not produced by hardwired logic. Two control memories are implemented and can be switched at run time: **vertical** (16-bit encoded microinstructions) and **horizontal** (25-bit, one bit per signal).
- **Optimized ALU:** one class, `RISC201_ALU`, performs every arithmetic, logic, shift, float and address operation. Adder, multiplier and divider algorithms can be selected (Kogge-Stone or ripple-carry, Booth radix-4 or shift-and-add, non-restoring or restoring).
- **Software suite:** two-pass assembler, disassembler, macro preprocessor for full-descending stack operations, static stack checker, runtime stack protection with exceptions, interactive CLI simulator and debugger, ASCII stack visualizer, evaluation script and automatic tests.
- **Requirements:** Python 3.8 or newer, no extra packages. Runs on Windows, macOS and Linux.
- **Related documents** (folder `docs/`): `SimpleRISC_Extended_ISA.docx` (instruction set), `Pipeline_4Stage.docx` and `Pipeline_6Stage.docx` (pipeline designs), `evaluation_report.md` (results, generated by `evaluate.py`).

A. Where each assignment item is implemented

Item	Requirement	Where it is met
1a	4-stage and 6-stage RISC201, compare their behaviour	Pipeline documents, `src/simulator.py` (`Pipeline4`, `Pipeline6`), section 13 of this document, CLI `compare`
1b	Microprogrammed control unit	`src/vertical_microprogramming.py`, `src/horizontal_microprogramming.py` (section 8)
1c	ALU with efficient arithmetic and logic	`src/alu.py` (section 9)
2a	Two-pass assembler to binary or hexadecimal	`src/assembler.py` (section 5)
2b	Disassembler from binary or hexadecimal words	`src/disassembler.py` (section 5)
2c	Interactive CLI: register file, instruction pipeline	`src/cli.py`: `regs`, `pipe`, `timeline`, `trace` (section 11)
2d	Step-by-step simulator with an interactive debugger	`src/cli.py`: `step`, `stepi`, `next`, `finish`, breakpoints, watchpoints (section 11)
2e	Preprocessor macros for full-descending stack operations	`src/preprocessor.py` (section 6)
2f	Stack boundary verification, runtime exceptions and memory protection	`src/stack_safety.py`, exceptions and `protect` modes in `src/simulator.py` (section 10)
2g	ASCII stack memory frame renderer	`src/stack_visualizer.py`, CLI `stack` and `frames` (section 11)
3	Microprogrammed control inside the simulator, microPC display, exceptions; both horizontal and vertical, compared	Decode stage reads the control memory; CLI `pipe`, `micro on`, `ucode`, `rom`, `upc`, `micro-stats`; `MicrocodeError` (sections 8 and 13)
4	Optimized ALU class drives all arithmetic, shift and logic	Every execute-stage operation calls `RISC201_ALU.execute` (section 9)
5	Evaluation: 4- vs 6-stage, programs, horizontal vs vertical, ALU algorithms	`evaluate.py`, `docs/evaluation_report.md` (section 13)

2. Getting started (how to run)

A. Requirements

- Python 3.8 or newer. Check with `python --version` (on macOS and Linux use `python3` in every command below).
- Nothing to install: the project uses only the Python standard library.
- Unzip the project and open a terminal in the `RISC201` folder. All commands below are run from there.

B. Run a first program

Start the interactive simulator with a program, run it and look at the registers:

```
python src/cli.py programs/01_factorial.asm
(risc201) run
[halted]  cycle 77, 39 retired, PC = 0x001C, CPI 1.974
(risc201) reg r2
r2 = 0x00375F00  3628800  (5.085032e-39)
(risc201) quit
```

`r2` holds 10! = 3628800. Type `help` for all commands and `help <command>` for one of them.

C. Run commands without the prompt (batch mode)

```
python src/cli.py programs/04_gcd.asm -x "run; regs; stats"
python src/cli.py programs/13_overflow_survive.asm -b scripts/debug_overflow_demo.txt
python src/cli.py programs/04_gcd.asm -m 4stage -c horizontal -x "run; stats"
```

`-x` takes commands separated by `;`, `-b` reads one command per line from a file. Options are listed in section 11.

D. Assemble to a hex or binary file

```
python src/assembler.py programs/02_fibonacci_recursive.asm
python src/assembler.py programs/04_gcd.asm -f bin -o gcd.bin -l gcd.lst -s
python src/disassembler.py programs/04_gcd.hex --annotate
```

The first command writes `programs/02_fibonacci_recursive.hex` (macros and `.org` are expanded first). The second writes binary words, a listing and the symbol table; the third turns a hex file back into assembly.

E. Run the tests and the evaluation

```
python tests/run_all.py      # every test suite and the evaluation, about 2 minutes
python evaluate.py           # only the evaluation; writes docs/evaluation_report.md
```

A successful run ends with `ALL TEST SUITES PASSED` and `ALL EXPECTATIONS MET`.

F. Write your own program

Create a file such as `programs/mytest.asm`, end it with `halt`, then load it in the CLI:

```
        mov   r1, 5          ; r1 = 5
        mov   r2, 7
        add   r3, r1, r2     ; r3 = 12
        halt

python src/cli.py programs/mytest.asm -x "run; reg r3"
```

Lines such as `; EXPECT r3 = 12` at the top of a program let `evaluate.py` check the result automatically (section 12).

3. Project structure

Path	Contents
`README.md`	Short guide: first run, commands, programs, key numbers
`evaluate.py`	Runs every program in every configuration, checks results, writes the evaluation report
`src/isa.py`	Instruction fields, opcode table, mnemonics, register names (shared by the tools)
`src/alu.py`	`RISC201_ALU`: adders, multipliers, dividers, barrel shifter, IEEE-754 float unit, converter

Path	Contents
src/assembler.py	Two-pass assembler: hex / binary output, listing, symbol table
src/disassembler.py	Hex / binary words back to re-assemblable assembly
src/preprocessor.py	Macro expansion, stack macros, .org, ; CONFIG lines
src/vertical_microprogramming.py	Vertical control memory (28 x 16 bits), dispatch table, field decoding
src/horizontal_microprogramming.py	Horizontal control memory (28 x 25 bits), dispatch table
src/simulator.py	Functional, 4-stage and 6-stage processor models, exceptions, stack protection
src/stack_safety.py	Static stack checker (push/pop balance, depth, recursion)
src/stack_visualizer.py	ASCII stack frame renderer and call-frame tracker
src/cli.py	Interactive CLI simulator and debugger
programs/	17 example programs, each with ; EXPECT result lines
scripts/	Debugger command scripts (batch mode)
tests/	run_all.py, ALU, toolchain and simulator tests, reference model pipeline_sim.py, hazard_check.py
docs/	Assignment, ISA and pipeline documents, this document, evaluation report

4. Instruction set summary

The full definition is in SimpleRISC_Extended_ISA.docx. This section summarizes what is needed to write programs.

- **Registers:** 16 x 32-bit, r0 to r13 general purpose, r14 = sp (stack pointer), r15 = ra (return address). Integers and floats share the registers.
- **Memory:** instruction and data memory are separate. Instructions are 32 bits, PC advances by 4.
- **Flags:** E (equal) and GT (greater than), written only by cmp / fcmp, read by beq / bgt.
- **Float:** IEEE-754 binary32. The F bit turns add sub mul div cmp mov min max abs neg into float operations.

A. Instruction formats (bit 31 = MSB)

Format	opcode	I	F	rd	rs1	Remaining bits
Register	[31:27]	0	F	[24:21]	[20:17]	rs2 [16:13], [12:0] unused
Immediate, integer	[31:27]	1	0	[24:21]	[20:17]	modifier [16:15], imm15 [14:0]
Immediate, float	[31:27]	1	1	[24:21]	[20:17]	imm17 [16:0] = top 17 bits of a binary32
Branch / call	[31:27]	-	-	-	-	offset [26:0], signed, in words
No operand	[31:27]	0	0	0	0	all other bits 0

B. Instructions

Opcode	Mnemonic	Operands	Meaning	Float form
00000-00011	add sub mul div	rd, rs1, rs2/imm	arithmetic	fadd fsub fmul fdiv
00100	mod	rd, rs1, rs2/imm	remainder	-
00101	cmp	rs1, rs2/imm	set flags E, GT	fcmp
00110 00111	and or	rd, rs1, rs2/imm	bitwise logic	-
01000 01001	not mov	rd, rs2/imm	invert / copy	fmov (mov only)
01010-01100	lsl lsr asr	rd, rs1, rs2/imm	shifts	-
01101	nop	-	no operation	-
01110 01111	ld st	rd, imm[rs1]	load / store word	-
10000-10010	beq bgt b	label	conditional / unconditional branch	-
10011 10100	call ret	label / -	call (ra = PC + 4) / return	-

Opcode	Mnemonic	Operands	Meaning	Float form
10101 10110	push pop	rd	full-descending stack	-
10111 11000	(free)	-	reserved, execute as nop	-
11001 11010	halt rte	-	stop / return from exception	-
11011-11110	min max abs neg	rd, rs1, rs2/imm (abs, neg: rd, rs2)	signed min / max, absolute, negate	fmin fmax fabs fneg
11111	fcvt	rd, rs2	int to float (`fcvt`) / float to int (`fcvti`)	F = direction

C. Assembly syntax

Element	Syntax	Example
Registers	`r0`-`r15`, `sp`, `ra`	`add r1, r2, sp`
Integer immediate	decimal, `0x` hex, `0b` binary, expressions; 15-bit signed	`mov r1, 0x7F`, `sub r1, r1, N-1`
Modifiers	`.u` zero-extend, `.h` shift left 16 (also `movu`, `movh`)	`mov.h r1, 0x7F00`
Float immediate	decimal float, top 17 bits are kept	`fmov r2, 1.5`
Memory operand	`offset[base]`	`ld r5, 4[r13]`, `st r1, 0[sp]`
Labels	`name:` at the start of a line; branches use the label	`loop: ... bgt loop`
Comments	`;`, `//` or `@` to end of line	`add r1, r1, 1 ; count`
Directives	`.equ NAME, value`, `.set`, `.word value`, `.text`, `.global`	`.equ N, 8`
Preprocessor	`.macro` / `.endm`, `PUSHM`, `POPM`, `ENTER`, `LEAVE`, `ALLOC`, `FREE`, `.org` (section 6)	`PUSHM {r1-r3}`

5. Assembler and disassembler

A. Two-pass assembler (src/assembler.py)

- **Preprocessing:** the command line first runs the preprocessor (macros and `.org`, section 6); `--no-preprocess` skips it.
- **Pass 1:** reads every line, records labels and `.equ` constants with their addresses, and checks mnemonics.
- **Pass 2:** encodes every instruction into a 32-bit word, resolves branch labels to PC-relative word offsets and checks operand ranges.
- **Diagnostics:** every error shows the file, line number and source line, with a suggestion for misspelled mnemonics or labels. All errors are reported in one run. Warnings (for example a float immediate that loses precision) can be turned into errors with `--strict`.

Option	Effect
`-o FILE`	output file (default: source name with .hex or .bin)
`-f hex` / `-f bin`	8 hex digits per line (default) or 32 binary digits per line
`-l FILE`	listing: address, word, source line
`-s`	print the symbol table
`--base ADDR`	address of the first instruction (default 0)
`--strict`	treat warnings as errors
`--no-preprocess`	assemble plain assembly only

Example listing (`-l`), lines refer to the original source file:

```
ADDRESS   WORD       LINE  SOURCE
00000000  4C20000A     5  mov   r1, 10
00000004  98000003     6  call  fib              ; r2 = fib(r1)
00000008  48A04000     7  mov   r5, r2
```

B. Disassembler (src/disassembler.py)

- Reads 8-digit hex words, `0x...`, `0b...` or 32-character binary strings, any number per line; `;`, `//` and `#` start comments.
- Branch targets inside the program get labels (`L_0008:`); `--no-labels` shows numeric offsets, `--annotate` adds the address and raw word as a comment.
- Output can always be assembled again: `assemble(disassemble(w)) == w` for every 32-bit word. A word that is not a valid instruction (reserved opcode, non-zero unused bits, F = 1 on an integer-only instruction) is written as `.word 0x...` with the reason as a comment.

```
python src/disassembler.py program.hex                # print assembly
python src/disassembler.py program.hex -o program.asm  # write it to a file
python src/disassembler.py --words 4C200005 C8000000   # mov r1, 5 / halt
```

6. Preprocessor and stack macros

`src/preprocessor.py` runs before the assembler (in the CLI, the simulator and `assembler.py`). The stack is **full-descending**: SP points at the last item pushed and the stack grows towards lower addresses.

A. Built-in stack macros

Macro	Expands to	Use
`PUSHM {r1, r4-r6}`	`push` of each register, highest number first	save several registers
`POPM {r1, r4-r6}`	`pop` of each register, lowest number first	restore them (same list as PUSHM)
`ENTER n`	`push ra`, then `sub sp, sp, n`	function prologue with n bytes of locals
`LEAVE n`	`add sp, sp, n`, then `pop ra`	function epilogue (follow with `ret`)
`ALLOC n` / `FREE n`	`sub sp, sp, n` / `add sp, sp, n`	reserve / release stack space

`PUSHM` leaves the lowest register at the lowest address (the same layout as ARM STMFD), so `POPM` with the same list restores every register. `sp` cannot be in the list, and `n` must be a multiple of 4.

B. User macros

```
.macro SUMTO dst, n=10          ; parameters, n has a default value
        mov   \dst, 0
        mov   r12, \n
loop\@: add   \dst, \dst, r12    ; \@ gives a unique label for each expansion
        sub   r12, r12, 1
        cmp   r12, 0
        bgt   loop\@
.endm
        SUMTO r5                ; r5 = 55
        SUMTO r6, 4             ; r6 = 10
```

C. Placement and program settings

- **.org ADDR** pads with `nop` up to byte address ADDR. It is used to place the exception handler at `EXEC_VECTOR` (`.org 0x400`).
- **; CONFIG name=value** lines set simulator options for a program, for example `; CONFIG stack_protect=trap`. The CLI applies them when the program is loaded, and a setting typed by the user always wins.

D. Errors

Errors inside a macro expansion are reported at the line where the macro was used, with the macro name, for example `prog.asm:6: error: expected a register ... (in expansion of macro BAD)`. Missing `.endm`, wrong argument counts, recursive macros and a `.org` that goes backwards are reported as errors. `python src/preprocessor.py prog.asm` prints the expanded source; the CLI command `source` shows it as well.

7. Processor models and pipelines

A. The three models

Model	Stages	Use
functional	one instruction per step	reference behaviour; fastest to run
4stage	IF, OF (decode + operand fetch), EX_MA (execute + memory), RW	design 1 (`Pipeline_4Stage.docx`)
6stage	IF, ID, OF, EX, MA, RW	design 2 (`Pipeline_6Stage.docx`), the CLI default

All three models use the same decoder (control memory) and the same `RISC201_ALU`, and always end with the same registers, memory, flags and exception state. They differ only in the number of cycles. Select the model with `-m` or the CLI command `mode`.

B. Timing

Event	4-stage	6-stage
Taken branch, call, ret	2 wasted cycles (resolved in EX_MA)	3 wasted cycles (resolved in EX)
Exception	2 wasted cycles	3 wasted cycles
rte	1 wasted cycle	1 wasted cycle
Load followed by a use of the loaded register	no stall (memory is read in EX_MA)	1-cycle load-use stall
Pipeline fill and drain	3 cycles	5 cycles
Execute latency	int mul 2, int div/mod 8, float add/sub/mul 3, float div 10, fcvt 2, everything else 1	same

C. Hazard handling

- **Forwarding:** 4-stage: the RW write data (W1, W2) is forwarded to EX_MA and bypassed into the OF register read. 6-stage: the EX_MA latch value and the RW write data are forwarded to EX (youngest first), and RW bypasses the OF register read.
- **Stalls:** a multi-cycle unit asserts `exBusy` and holds the earlier stages; the 6-stage pipeline also stalls one cycle for load-use (`stallLU`).
- **Flushes:** a taken branch or an exception kills the younger instructions; `halt` freezes fetch until older instructions finish.
- **Priority** when events coincide: exception > taken branch > rte > halt > stalls.

D. Memory map and parameters

Parameter	Value	Note
`STACK_BASE`	0x2000	SP starts here; pop may not move SP above it
`STACK_LIMIT`	0x1F00	push may not move SP below it (256-byte stack)
`EXEC_VECTOR`	0x400	exception handler address; a program shorter than 0x400 gets a `halt` there
Instruction memory	word per 4 bytes from address 0	fetching past the program returns `nop`
Data memory	32-bit words addressed by byte address	separate from instruction memory; starts empty (all 0)

The datapath diagrams follow on the next two pages (Figures 1 and 2).

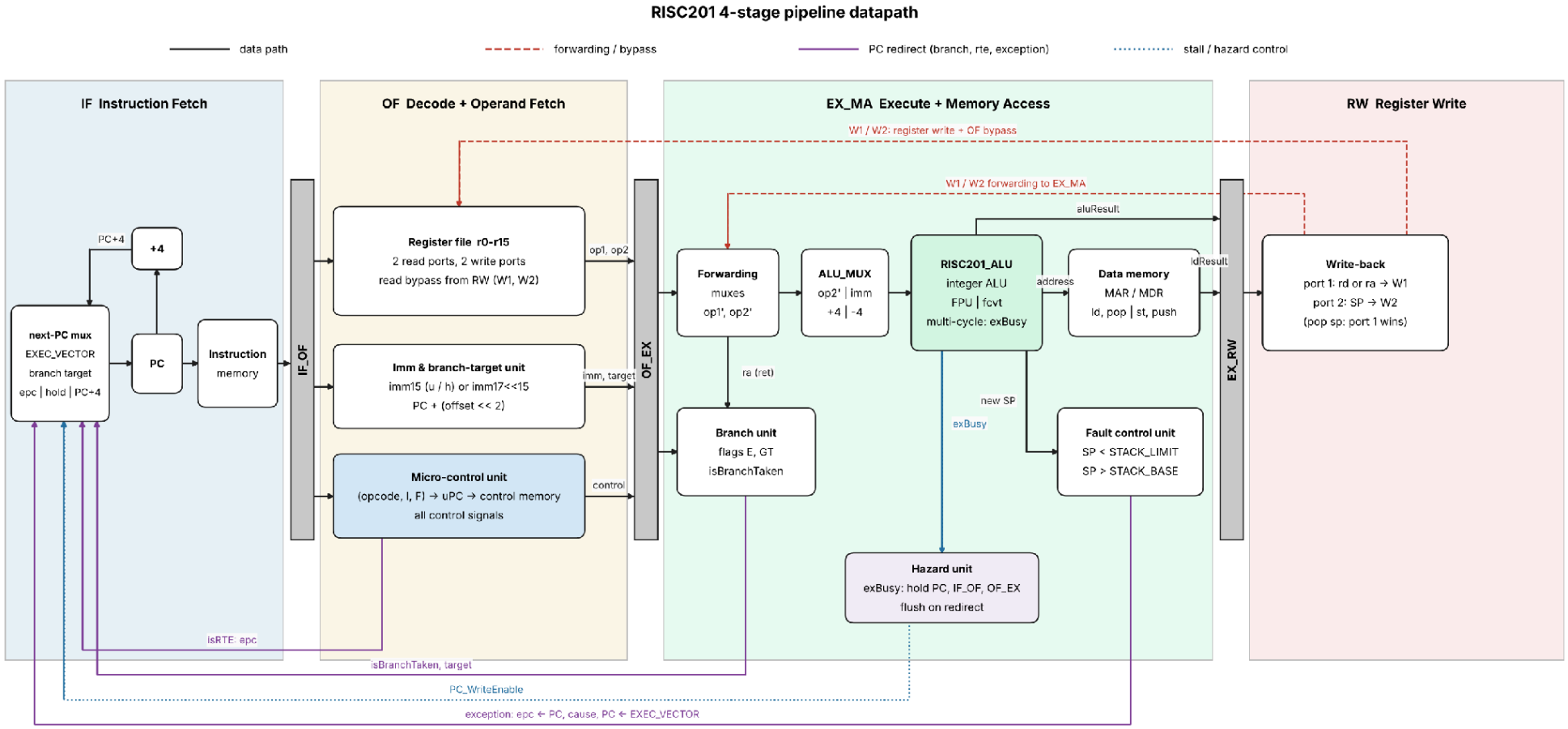


Figure 1. 4-stage pipeline datapath: latches, forwarding and bypass paths (red), PC redirects for branches, rte and exceptions (purple), stall control (blue).

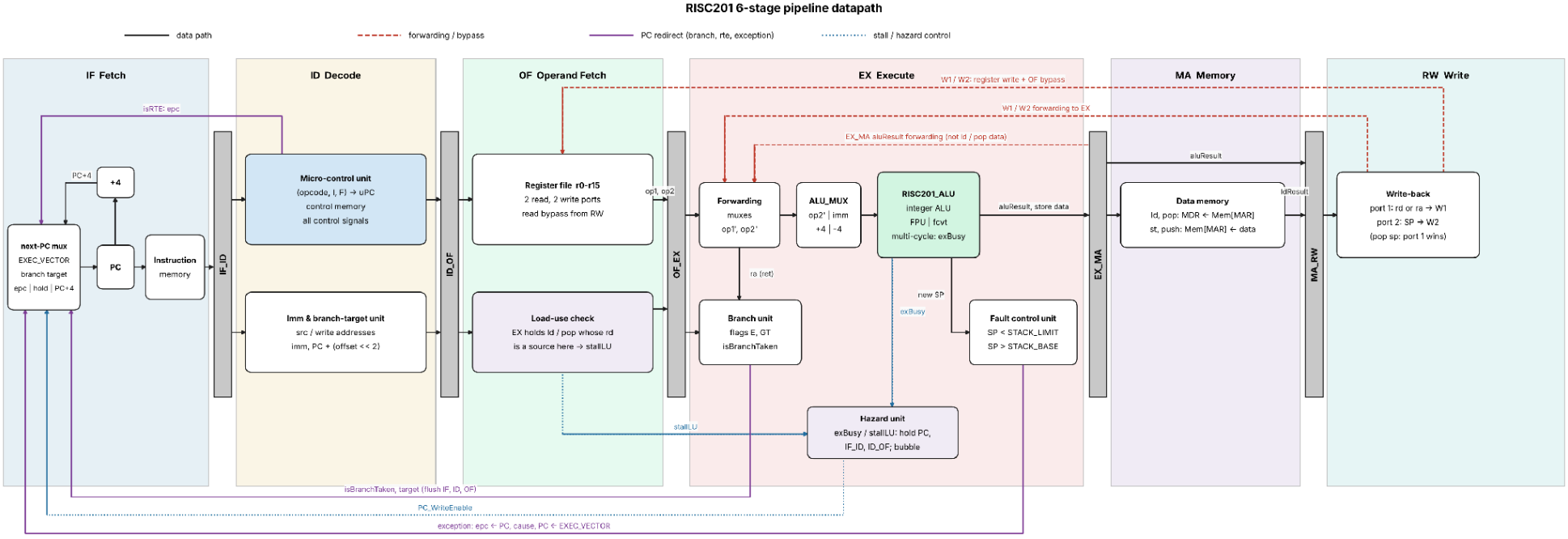


Figure 2. 6-stage pipeline datapath with EX_MA and RW forwarding, the OF bypass and the load-use check.

8. Microprogrammed control unit

A. How it works

- The decode stage (OF in the 4-stage, ID in the 6-stage pipeline) uses the opcode, I bit and F bit as the address of a **dispatch table**, which gives the start address of the microroutine: the **uPC**.
- The control memory word at that address is the **microinstruction**. It gives every control signal: ALU function, float or integer unit, register reads and writes, memory and stack operations, branches, rte and halt.
- Each RISC201 instruction is a one-microinstruction routine: a pipeline needs a new set of control signals every cycle, so the sequencer dispatches once per instruction and the uPC returns to 0x00.
- The F bit is part of the dispatch address. F = 1 selects the floating point microinstruction only for instructions that have a float form; for all other instructions F is ignored. No control decision is made outside the control memory.
- Two control memories with the same addresses are implemented. They give identical signals for all 128 combinations of opcode, I and F, and identical results and cycle counts on every program.

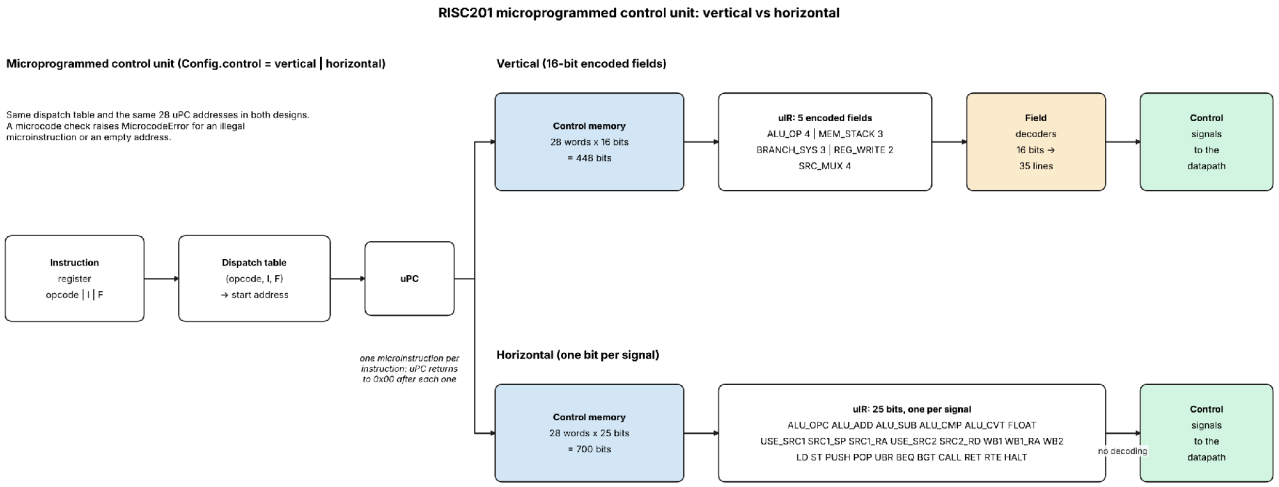


Figure 3. Microprogrammed control unit: dispatch, uPC, vertical and horizontal control memory.

B. Vertical microinstruction (16 bits, 5 encoded fields)

Field	Bits	Values
ALU_OP	4	none, opc, fopc, add, sub, cmp, fcmp, i2f, f2i
MEM_STACK	3	none, ld, st, push, pop
BRANCH_SYS	3	none, ubr, beq, bgt, call, ret, rte, halt
REG_WRITE	2	none, wb1 (rd or ra), wb2 (SP), both
SRC_MUX	4	n_n, r_r, r_i, r_rd, n_r, n_i, sp_r, sp_n, ra_n (which registers are read)

Five field decoders turn the 16 bits into 35 signal lines. `opc` sends the instruction's own opcode to the integer unit, `fopc` to the float unit; `add` and `sub` are fixed operations used for addresses, SP and the call link.

C. Horizontal microinstruction (25 bits, one per signal)

Group	Bits
ALU function	ALU_OPC, ALU_ADD, ALU_SUB, ALU_CMP, ALU_CVT, FLOAT
Operand select	USE_SRC1, SRC1_SP, SRC1_RA, USE_SRC2, SRC2_RD
Register write	WB1, WB1_RA, WB2
Memory and stack	LD, ST, PUSH, POP
Control flow and system	UBR, BEQ, BGT, CALL, RET, RTE, HALT

D. Control memory contents

uPC	Vertical microinstruction	Horizontal: bits set	Instructions
0x00	none|none|none|none|n_n	(none)	nop, reserved opcodes
0x10 / 0x11	opc|none|none|wb1|r_r / r_i	ALU_OPC USE_SRC1 (USE_SRC2) WB1	add sub mul div mod and or lsl lsr asr min max (reg / imm)
0x12 / 0x13	opc|none|none|wb1|n_r / n_i	ALU_OPC (USE_SRC2) WB1	not mov abs neg (reg / imm)
0x14 / 0x15	i2f|none|none|wb1|n_r / n_i	ALU_CVT (USE_SRC2) WB1	fcvt (int to float)
0x16 / 0x17	f2i|none|none|wb1|n_r / n_i	ALU_CVT FLOAT (USE_SRC2) WB1	fcvti (float to int)
0x18 / 0x19	fopc|none|none|wb1|r_r / r_i	ALU_OPC FLOAT USE_SRC1 (USE_SRC2) WB1	fadd fsub fmul fdiv fmin fmax
0x1A / 0x1B	fopc|none|none|wb1|n_r / n_i	ALU_OPC FLOAT (USE_SRC2) WB1	fmov fabs fneg
0x20	add|ld|none|wb1|r_i	ALU_ADD USE_SRC1 WB1 LD	ld
0x21	add|st|none|none|r_rd	ALU_ADD USE_SRC1 USE_SRC2 SRC2_RD ST	st
0x22	sub|push|none|wb2|sp_r	ALU_SUB USE_SRC1 SRC1_SP USE_SRC2 SRC2_RD WB2 PUSH	push
0x23	add|pop|none|both|sp_n	ALU_ADD USE_SRC1 SRC1_SP WB1 WB2 POP	pop
0x30 / 0x31	cmp|none|none|none|r_r / r_i	ALU_CMP USE_SRC1 (USE_SRC2)	cmp
0x3A / 0x3B	fcmp|none|none|none|r_r / r_i	ALU_CMP FLOAT USE_SRC1 (USE_SRC2)	fcmp
0x32 - 0x34	none|none|ubr / beq / bgt|none|n_n	UBR / BEQ / BGT	b, beq, bgt
0x35	add|none|call|wb1|n_n	ALU_ADD WB1 WB1_RA CALL	call
0x36	none|none|ret|none|ra_n	USE_SRC1 SRC1_RA RET	ret
0x37 / 0x38	none|none|rte / halt|none|n_n	RTE / HALT	rte, halt

28 words in each memory: 448 bits (vertical) and 700 bits (horizontal). The CLI command `rom` prints the full contents with the binary words.

E. Microcode exceptions

Both modules check the whole control memory when they are loaded, and every microinstruction again when it is used. `MicrocodeError` is raised for an unknown field value, a dispatch to an empty address, or an illegal horizontal word (two ALU functions, two src1 sources, two memory or two control-flow operations, FLOAT without an ALU function). The CLI reports it as `microcode exception in cycle N: ...` and stays usable.

F. Viewing the microcode in the CLI

Command	Shows
`pipe` / `step`	every pipeline stage with the uPC of its instruction
`micro on`	adds each stage's microinstruction to every `step` and `pipe` output
`ucode <addr|label>`	the microsequencer steps for one instruction (below)
`rom`, `upc <addr>`	the whole control memory, or one word in both memories
`micro-stats`	how often each microinstruction was used by the retired instructions
`control vertical|horizontal`	switch the control memory (alias `cu`)

```
(risc201) ucode recurse
instruction 0x0028 <recurse>: push r3   word 0xA8600000
control unit: vertical (16-bit microinstructions)
  step 1  IR  <- 0xA8600000         opcode=10101 (push)  I=0  F=0
  step 2  uPC <- DISPATCH[21, 0, 0] = 0x22
  step 3  uIR <- ROM[0x22] = sub|push|none|wb2|sp_r  (0b0100011000100110)
  step 4  decode the encoded fields: ALU_OP=sub, MEM_STACK=push,
          BRANCH_SYS=none, REG_WRITE=wb2, SRC_MUX=sp_r
          -> signals: isPush isWb2
```

```
         -> ALU function: sub;  src1=sp  src2=r3  write port2=sp
step 5   uPC <- 0x00 (end of microroutine; dispatch again for the next one)
```

9. ALU

A. Interface

`RISC201_ALU(adder_mode, mul_mode, div_mode).execute(opcode, op1, op2, is_float, is_fcvt)` returns `(result, flag_E, flag_GT, latency)`. The execute stage calls it for every operation, including address calculation, SP plus or minus 4 and the call link PC + 4, with the function chosen by the microinstruction. The returned latency drives the multi-cycle stall.

B. Algorithms

Unit	Algorithm	Work for 32-bit operands
Adder	Kogge-Stone parallel prefix (default)	5 prefix levels, 7 levels in total, 129 prefix cells
Adder	Ripple-carry	32 full adders in series
Multiplier	Booth radix-4 (default)	16 partial products, each 0, +-1x or +-2x
Multiplier	Shift-and-add	32 steps, one add per 1 bit of the multiplier
Divider	Non-restoring (default)	32 add/subtract steps and one final correction
Divider	Restoring	32 subtract steps plus an add-back after every negative step
Shifter	Barrel shifter	5 multiplexer levels (lsl, lsr, asr)
Float add, sub	align, add, normalize, round (IEEE-754 binary32)	smaller operand shifted right by the exponent difference with a sticky bit; significands added or subtracted with the selected adder
Float mul	24 x 24 significand product	built by the selected multiplier: 13 Booth digits or 24 shift-and-add steps, added as two chained 32-bit adder passes
Float div	significand long division	27 quotient bits from the selected divider (non-restoring or restoring); a non-zero remainder sets the sticky bit
Float rounding	guard, round and sticky bits	round to nearest, ties to even; subnormals flushed to zero; overflow gives Inf

The float unit works directly on the bit fields: it unpacks sign, exponent and the 24-bit significand (hidden 1 made explicit), operates on the significands with the integer units above, normalizes, rounds and packs the result. It does not use the host computer's floating-point arithmetic. Exponents are 8-bit values and are handled with ordinary integer arithmetic. The float latencies in part C are the timing figures of the design: the float divider, for example, works out 27 quotient bits one at a time, which a real 10-cycle divider would do with a higher-radix method.

C. Latencies (cycles in the execute stage)

Operation	Cycles
Integer add, sub, logic, shifts, cmp, mov, min, max, abs, neg, address calculation	1
Integer mul	2
Integer div, mod	8
Float add, sub, mul	3
Float div	10
fcvt (both directions)	2
Float cmp, min, max, abs, neg, mov	1

D. Special cases (no exception is raised)

Case	Result
Integer overflow (e.g. INT_MAX + 1)	wraps around (two's complement)

Case	Result
Integer divide by zero	`div` gives 0xFFFFFFFF, `mod` gives the dividend
INT_MIN / -1	INT_MIN, remainder 0
Float overflow, x / 0	+Inf or -Inf
0 / 0, Inf - Inf, NaN operands	canonical quiet NaN 0x7FC00000
Subnormal input or result	flushed to signed zero
fcvti of NaN / out-of-range value	0 / saturates to INT_MAX or INT_MIN
fcmp with a NaN operand	both flags 0, so `beq` and `bgt` are not taken
fmin / fmax with one NaN operand	the other operand

E. Choosing algorithms

In the CLI: `alu adder ripple`, `alu mul shiftadd`, `alu div restoring` (`alu` alone shows the current choice and the add/subtract steps done so far). On the command line: `--adder`, `--mul`, `--div`. The choice also applies to the float unit: float add and subtract use the selected adder, float multiply the selected multiplier and float divide the selected divider. All combinations give the same results and the same cycle counts; section 13 compares the work each one does.

10. Exceptions and stack safety

A. Exception framework

Cause	Name	Raised by
1	stack overflow / underflow	`push` that would move SP below STACK_LIMIT, `pop` that would move SP above STACK_BASE
2	stack protection violation	only with `protect trap` (part C)

- Exceptions are **precise**: they are detected in the execute stage, the faulting instruction writes nothing, younger instructions are flushed and older ones finish.
- `epc` receives the address of the faulting instruction, `cause` the cause code, and the PC jumps to `EXEC_VECTOR` (0x400).
- The handler can fix the problem and execute `rte` to run the faulting instruction again, branch somewhere else, or `halt`. Exceptions inside the handler are not supported.
- A program that does not reach 0x400 gets a `halt` there, so an unexpected exception stops the program with `epc` and `cause` set (CLI command `exc`).

```
        mov   sp, 0x1F00        ; stack full: SP at STACK_LIMIT
        mov   r1, 5
        push  r1                ; exception, epc = 0x08, cause = 1
        pop   r2                ; runs after the handler's rte retried the push
        halt
.org 0x400
handler:
        mov   sp, 0x1F80        ; give the program room on the stack
        rte                     ; PC <- epc
```

B. Static stack checker (src/stack_safety.py)

Runs on every `load` in the CLI (summary) and with the command `check` (details). Every function (main, each call target and the handler) is followed along all of its paths while counting the bytes on the stack.

Level	Finding
error	pop or add sp below the function's entry depth (corrupts the caller's frame, or underflows in main)
error	ret with bytes still pushed (unbalanced push / pop)
error	two paths reach the same instruction with different depths (unbalanced branches, or a loop that keeps pushing)
warning	worst-case depth through the call graph is larger than the stack (256 bytes)
warning	recursion: depth cannot be bounded statically (the runtime check guards it)

Level	Finding
note	sp written in a way the checker cannot follow (for example `mov sp, ...`)

C. Runtime stack memory protection

Mode	Behaviour
`off` (default)	the documented design: only push and pop are checked
`warn`	violations are reported as `STACK WARNING` and the program continues
`trap`	a violation raises a precise exception with cause 2 before anything is written

Rules checked in `warn` and `trap` mode: `ld` / `st` into the stack region must use `sp` as the base register and stay inside [SP, STACK_BASE), and an ALU instruction that writes `sp` must keep it inside [STACK_LIMIT, STACK_BASE]. Select the mode with `--protect` or the CLI command `protect`; programs 10 and 17 select `trap` with a `; CONFIG` line.

11. CLI simulator and debugger

A. Starting the CLI

```
python src/cli.py [program] [options]
```

Option	Meaning
`program`	.asm / .s source (preprocessed and assembled) or .hex / .bin image
`-m MODE`	functional, 4stage or 6stage (default 6stage)
`-c CONTROL`	vertical or horizontal (default vertical)
`--adder`, `--mul`, `--div`	ALU algorithms (section 9)
`--protect off|warn|trap`	stack protection (default off, or the program's `; CONFIG` line)
`-x "cmd; cmd"`	run these commands and exit
`-b FILE`	run the commands in a file (one per line, `#` comments) and exit

B. Command reference

Group	Command	What it does
Program	`load <file>`, `reload`	load (and assemble) a program; reload after editing
	`list [addr] [n]`, `disasm`	disassemble around an address / the whole program
	`source`, `symbols`, `check`	expanded source, symbol table, static stack check
Machine	`mode`, `control` (`cu`)	processor model, control memory
	`alu`, `adder`, `protect`, `reset`	ALU algorithms, stack protection, restart from PC 0
Execution	`step [n]` (`s`)	advance n clock cycles and show the pipeline
	`stepi [n]` (`si`)	run until n more instructions have finished
	`next` (`n`), `finish`	step over a call / run until the current function returns
	`run [max]`, `continue` (`c`), `until <addr>`	run to halt, breakpoint, watchpoint or address
Breakpoints	`break <addr|label>` (`b`), `breaks`, `breakpoints`	set and list breakpoints
	`delete [n|all|addr]`, `clear-breaks`	remove breakpoints
	`watch <reg>`, `watch mem <addr>`, `unwatch`	stop when a register or memory word changes
State	`regs`, `reg <r> [value]`	register file (hex, signed, float), read or set one register
	`mem [addr] [n]`, `setmem <addr> <value>`	data memory

Group	Command	What it does
	`exc`, `stats`	exception state; cycles, CPI, stalls, flushes
	`print-regs`, `print-mem`, `print-stack`, `print-pipeline`	aliases of regs, mem, stack and pipe
Pipeline	`pipe`, `timeline [first] [last]`, `trace on|off`	stages of the last cycle, cycle diagram, print every cycle
Microcode	`micro on|off`, `ucode`, `rom`, `upc`, `micro-stats`	section 8
Stack	`stack [words]`, `frames` (`bt`)	ASCII stack picture, call stack
Evaluation	`compare`	run the program in all 3 models x 2 control units, compare cycles
Other	`help [command]`, `quit` (`q`, `exit`)	help, leave

Breakpoints stop just after the instruction at the address has finished (the one point that is well defined in every model). Commands with a hyphen can also be typed with an underscore.

C. Example debugging session

```
python src/cli.py programs/02_fibonacci_recursive.asm
(risc201) break recurse
(risc201) run
(risc201) frames
(risc201) stack
(risc201) step
(risc201) watch r5
(risc201) delete all
(risc201) run
(risc201) stats
```

This session stops in the recursive case (`break recurse`, `run`), shows the call stack and the picture of the frames, advances one clock cycle with every stage and its uPC, then removes the breakpoint and runs until the result is written to `r5` (`watch r5`).

D. Pipeline view and timeline

```
cycle 9
  IF     0x0020  mul r7, r6, r1
  ID     0x001C  add r6, r5, r5 [uPC: 0x10]
  OF     0x0018  ld r5, 0[r13] [uPC: 0x20]
  EX     0x0014  st r4, 0[r13] [uPC: 0x21]
  MA     0x0010  add r4, r3, r2 [uPC: 0x10]
  RW     0x000C  add r3, r2, r1 [uPC: 0x10]
  * retire 0x000C add r3, r2, r1  [r3=0x00000015]
  * forward to EX (st r4, 0[r13]): r4 from EX_MA
```

`timeline` prints an instruction x cycle diagram. A suffix marks a stall (`:s`), a busy multi-cycle unit (`:b`), an instruction killed by a flush (`:x`) and a faulting instruction (`:!`).

E. Stack visualizer

```
STACK  full-descending   base 0x2000   limit 0x1F00   SP 0x1FE0
used 32 / 256 B  [####........................]  12%   frames: 3 (+main)
0x2000    |=========== STACK_BASE ===========|
          +==== main
          +==== fib  (call @0x0004 -> returns to 0x0008)
0x1FFC    | 0x00000008 |           8 |  saved return address? (fib)
0x1FF8    | 0x00000000 |           0 |
0x1FF4    | 0x0000000A |          10 |
          +==== fib  (call @0x0034 -> returns to 0x0038)
0x1FF0    | 0x00000038 |          56 |  saved return address? (fib)
  ...
0x1FE0    | 0x00000000 |           0 |  <- SP
0x1FDC    |    free    |             |
          |   ... 216 B free ...
0x1F00    |=========== STACK_LIMIT ==========|
```

Frames are found by following `call` and `ret` as they finish; words equal to a return address are marked, and frames abandoned by an exception handler are removed.

12. Example programs

Every program in `programs/` starts with `; EXPECT` lines (for example `; EXPECT r5 = 55`, `; EXPECT mem[0x100] = 1`, `; EXPECT cause = 1`) that `evaluate.py` and the tests check in every configuration.

Program	Shows	Result
01_factorial	loop with the 2-cycle multiplier	r2 = 3628800
02_fibonacci_recursive	recursion with ENTER, LEAVE, PUSHM, POPM	fib(10) = 55
03_bubble_sort	memory, nested loops, load-use hazards	8 words sorted
04_gcd	mod on the 8-cycle divider	gcd(1071, 462) = 21
05_float_newton_sqrt	float unit, fcvt	1000 x sqrt(2) = 1414
06_hazards	forwarding, load-use and multi-cycle hazards	r11 = 71
07_macros	user macros, default parameters, unique labels	r5 = 55, r6 = 10
08_stack_overflow	overflow exception, handler halts	cause 1 after 64 levels
09_exception_rte	precise exception, handler fixes SP, rte retries	r2 = 5
10_stack_protection	stores that corrupt the stack (protect trap)	cause 2
11_all_units_demo	every functional unit	r9 = 45
12_stack_underflow_recover	pop on an empty stack, handler recovers	r2 = 99
13_overflow_survive	overflow, handler recovers and continues	r5 = 777
14_int_overflow_silent	integer overflow wraps, detected in software	r7 = 0x7FFFFFFF
15_float_overflow_underflow	Inf, 0, NaN without exceptions	r6 = NaN
16_unbalanced_stack	push without pop, found by `check`	SP = 0x1FEC
17_exception_suite	all exception tests, results at 0x200	4 exceptions

13. Evaluation

`python evaluate.py` runs every program in 12 configurations (3 processor models x 2 control units x 2 adders) and in all 8 combinations of ALU algorithms, checks all `; EXPECT` lines, and writes `docs/evaluation_report.md`. Result: **0 failures**, and all configurations of a program end in exactly the same state.

A. 4-stage vs 6-stage pipeline

Program	Instr.	4-stage cycles	4-stage CPI	6-stage cycles	6-stage CPI	Load-use stalls (6)
01_factorial	39	67	1.718	77	1.974	0
02_fibonacci_recursive	1858	2745	1.477	3366	1.812	177
03_bubble_sort	350	463	1.323	548	1.566	28
04_gcd	24	56	2.333	62	2.583	0
05_float_newton_sqrt	44	129	2.932	135	3.068	0
06_hazards	16	34	2.125	38	2.375	2
08_stack_overflow	197	333	1.690	401	2.036	0
17_exception_suite	326	561	1.721	656	2.012	2
All 17 programs	**3253**	**5087**	**1.564**	**6105**	**1.877**	

- The 6-stage pipeline needs **20.0% more cycles** (CPI 1.877 compared with 1.564).
- Reason 1: a taken branch, call or ret wastes 3 cycles instead of 2, because the branch is resolved one stage later.
- Reason 2: a value loaded by `ld` or `pop` cannot reach the next instruction in time, which costs a 1-cycle load-use stall. The 4-stage pipeline reads memory in EX_MA and never waits.
- Multi-cycle units stall both pipelines by the same amount, and the 6-stage pipeline needs 2 extra cycles to fill and drain.

- Each 6-stage stage does less work, so its clock can be faster: if the 6-stage clock is at least 17% faster it finishes these programs sooner (time = cycles x clock period).

B. Vertical vs horizontal control unit

	Vertical	Horizontal
Control memory	28 words x 16 bits = 448 bits	28 words x 25 bits = 700 bits
Microinstruction	5 encoded fields	one bit per control signal
Hardware after the memory	5 decoders (16 bits to 35 lines)	none
Dispatch	opcode, I, F	same
Results and cycles	identical	identical

The vertical design needs 36% fewer bits but adds decoder delay in the decode stage; the horizontal design is wider but its signals are ready as soon as the word is read, and a new signal combination needs no decoder change.

C. ALU algorithms

Algorithm	Average add/sub steps per operation	Worst case seen
Booth radix-4 multiplier	12.0	16
Shift-and-add multiplier	16.1	26
Non-restoring divider	32.6	33
Restoring divider	57.6	64

- Measured on 4,000 random 32-bit operand pairs (the dividers also on 4,000 pairs with a small divisor).
- Booth needs about 25% fewer steps than shift-and-add and never more than 16. In the test programs, which multiply by small numbers, shift-and-add did slightly fewer steps (25 compared with 28).
- The non-restoring divider needs about 43% fewer steps than the restoring one, because it never adds the divisor back.
- Kogge-Stone needs about 7 logic levels against 32 carry stages for ripple-carry. All 8 combinations of adder, multiplier and divider give the same results and cycle counts, because the latency of each operation is fixed by the design.

14. Testing and verification

Suite	What it checks	Size
`tests/test_alu.py`	`RISC201_ALU` against reference arithmetic, all 8 algorithm combinations, special values, latencies	640,000 operations
`tests/test_toolchain.py`	assembler / disassembler round trip, preprocessor, stack checker, stack protection, control memories, visualizer, CLI commands, standalone assembler, every program	247 checks
`tests/test_simulator.py`	simulator against the independent reference model `pipeline_sim.py`: final state and exact cycle count, both pipelines, both control units	26 directed, 669 hazard, 1,600 random programs
`evaluate.py`	every `; EXPECT` line of every program in every configuration	17 programs x 20 configurations

- `python tests/run_all.py` runs all suites (about 2 minutes); every suite passes.
- The reference model `tests/pipeline_sim.py` and `tests/hazard_check.py` verify the pipeline designs themselves, including fault-injection runs that prove the checks can detect broken forwarding, stalls and priorities.

15. Design decisions and limitations

Issue	Decision
One microinstruction per instruction	A pipeline needs fresh control signals every cycle, so each instruction is a one-word microroutine and the uPC returns to 0x00.
Latency vs ALU algorithm	Execute latencies are fixed by the design documents. The algorithm changes the hardware work inside those cycles, not the cycle count.
Stack protection off by default	The documented design only checks push and pop. Protection is an optional extension (`warn`, `trap`) so the default behaviour matches the design documents.
Exceptions	Precise, one level. Nested exceptions are not supported; the handler must change SP or halt before `rte`, or the same push / pop faults again.
Breakpoints	Stop after the instruction at the address has finished, which is well defined in all three models.
Data memory	Words are stored by byte address; alignment is not checked. Data memory and instruction memory are separate.
Program size	Code below 0x400 can coexist with a handler placed by `.org 0x400`; a program that does not reach 0x400 gets a default `halt` handler.
Constants	Immediates are 15 bits (integer) or the top 17 bits of a float; larger constants are built with `mov.h` plus `or`, or loaded with `ld`.